CLAUDE.md — CorporateMind AI

> *Operating manual for engineers (human + AI) working on **CorporateMind AI** — an autonomous AI corporate-outreach and multi-channel growth OS for trainers, coaches, consultants, and speakers.*

This file is the **thin master**. Detailed rules live in `.claude/rules/*.md` and are loaded **on demand** (when working in that area). Reusable workflows live in `.claude/skills/*/SKILL.md` and are invokable as slash commands.

Role

Act as a staff engineer, AI architect, and SaaS production reviewer. Production-grade code only — no demo scaffolding, no quick hacks, no half-finished implementations.

Engineering Philosophy

These are the tiebreakers when a specific rule doesn't cover your situation:

1. **Correctness over speed** — A slow correct campaign beats a fast wrong one. Our trainers' reputations depend on what we send.
2. **Simplicity over premature abstraction** — Stage 1 is a modular monolith by design. No Kafka, no K8s until explicit scaling triggers are hit (PRD §23).
3. **Explicitness over hidden magic** — Tenant isolation, Euri routing, ComplianceGuard — all explicit, all visible, all testable.
4. **Stability over cleverness** — The platform runs autonomously while trainers sleep. Boring reliable code beats clever fragile code.
5. **Governance over chaos** — Compliance, audit logs, HITL gates, and cost ceilings are not optional. They're how we stay in business.

Stack at a glance

- **Backend:** FastAPI (async) — modular monolith under `apps/api/src/corpmind/modules/`
- **Frontend:** Next.js 14 App Router — under `apps/web/`
- **Data:** PostgreSQL (RLS for tenancy), Redis (cache/queue/pubsub), Qdrant (embeddings + semantic cache), Meilisearch (BM25), Cloudinary/S3
- **AI runtime:** LangGraph multi-agent under `apps/api/src/corpmind/agents/`
- **AI gateway: Euri AI Gateway is the sole egress to LLM providers.** Use `corpmind.ai.euri_client.EuriClient`. Direct imports of `openai` / `anthropic` /etc. are blocked by `.claude/scripts/block-direct-llm-imports.sh`.
- **Workers:** Celery (Redis broker) + Celery beat
- **Channels:** Email, WhatsApp Business Cloud, Telegram, Instagram, Facebook, LinkedIn-post (public only — never personal DMs)
- **Deploy:** Vercel (web) + Railway (api/workers/db/redis)
- **Observability:** Langfuse (LLM traces), Prometheus + Grafana, Sentry, OpenTelemetry

The 7 pillars

`trainer_intel` | `hr_discovery` | `outreach` | `social` | `proposals` | `crm` | `multi_agent_runtime` Each module under `apps/api/src/corpmind/modules/<name>/` follows Ports & Adapters: `api.py` | `service.py` | `repo.py` | `models.py` | `schemas.py` | `events.py` **Inter-module rule:** modules NEVER import each other's `repo.py` or `models.py`. Cross-module communication is via service interfaces (DI) or the event bus.

When working on X, read `.claude/rules/X.md`

Working on…	Read these rules
Backend modules, services, repos	`backend-python.md` + `multi-tenancy.md`
Frontend pages, components, hooks	`frontend-nextjs.md`

Any LLM call	`euri-gateway.md` + `prompt-engineering.md`
New / modified agent or workflow	`langgraph-agents.md` + `prompt-engineering.md`
Channel adapter or webhook	`channel-adapter.md` + `compliance-guard.md` + `security.md`
RAG / retrieval pipeline	`rag-retrieval.md`
Outbound message (any channel)	`compliance-guard.md`
Database migration	`deployment-guardrails.md` (expand-then-contract) + `multi-tenancy.md`
Anything customer-facing	`compliance-guard.md` + `security.md`
Celery task	`queue-celery.md`
Analytics / dashboards	`analytics.md` + `observability.md`
Admin tooling	`admin-panel.md` + `security.md`
Hindi / regional language	`multi-language.md`
Autonomous schedules / cron	`automation.md`
Cost-sensitive change	`ai-cost-governance.md`
Before any PR merges	`testing.md` + `deployment-guardrails.md` + `observability.md`
Architectural decision	`adr.md`
Risky / user-visible rollout	`feature-flags.md`
Incident in progress	`incident-response.md`

Invokable skills (slash commands)

Skill	When to use
`/create-module`	Scaffold a new business module
`/create-api`	Add a REST endpoint to an existing module
`/create-langgraph-agent`	Add a new agent (requires ADR)
`/create-langgraph-workflow`	Add a multi-step workflow with checkpoints + HITL
`/create-channel-adapter`	Add a new channel (WA/TG/IG/FB/LI/Email/...)
`/create-webhook-handler`	Add an inbound webhook with HMAC + replay protection
`/create-rag-pipeline`	Add a new retrieval pipeline for a source type
`/create-migration`	Generate an Alembic migration (expand-then-contract)
`/create-test-suite`	Scaffold unit/repo/api/integration/isolation tests
`/create-prompt-template`	Add a versioned prompt with fixtures + evals
`/create-compliance-rule`	Add a non-bypassable rule to ComplianceGuard
`/create-analytics-metric`	Add a metric (definition, rollup, panel, alert)

`/review-security`	Focused security review on changed files
`/review-tenant-isolation`	Verify tenant_id propagation + generate regression test
`/debug-workflow-replay`	Replay a failed agent run deterministically against staging

---

Core behavior

- Production-grade over quick hacks. No fake/demo scaffolding.
- Preserve existing patterns; don't refactor opportunistically.
- Inspect neighbors before editing — match local style.
- Briefly justify tradeoffs when introducing new patterns.
- Design for horizontal scale and per-tenant isolation from day one.
- Stage 1 = modular monolith on Railway. Do **not** preemptively introduce Kafka, K8s, or microservices — those have explicit migration triggers (see PRD §23).

Hard invariants (P0)

1. **Tenant isolation.** Every business table has `tenant_id`. RLS enabled. `TenantContext` propagated. Cross-tenant leaks are P0 bugs.
2. **Euri-only egress.** No direct LLM-provider SDK imports outside `apps/api/src/corpmind/ai/`. Mechanically enforced.
3. **ComplianceGuard before every send.** No bypass. Audit-logged on block.
4. **No LinkedIn personal-DM automation. Ever.** Public company-page posts only.
5. **Migrations are expand-then-contract.** Never deploy a destructive contract step in the same release as the expand.
6. **Webhooks verify HMAC before parsing.** Replay-protected.

Pre-PR checklist

Before opening a PR, verify:

- [ ] Module boundaries respected (no cross-module `repo` / `models` imports)
- [ ] `tenant_id` propagated and filtered on every query
- [ ] LLM calls via `EuriClient`; prompts versioned in `ai/prompts/`
- [ ] Outbound messages pass through `ComplianceGuardAgent`
- [ ] Idempotency keys on mutating endpoints
- [ ] Edge cases handled (empty, timeouts, rate limits, partial failures)
- [ ] Structured logs + Langfuse trace for important flows
- [ ] Tests added/updated; tenant-isolation test if new table
- [ ] No PII in logs
- [ ] No direct provider SDK imports
- [ ] Migration reversible; no destructive change without backup path
- [ ] Feature-flagged if user-visible
- [ ] ADR written / referenced if architectural

Reference documents

- **Full PRD:** docs/PRD.md — investor-grade product specification (v3.0, 34 sections + appendices)
- **ADRs:** `docs/adr/NNNN-*.md`
- **Runbooks:** `ops/runbooks/*.md`

Hooks (automatic)

- **PreToolUse (Edit|Write):** `.claude/scripts/block-direct-llm-imports.sh` — blocks direct LLM-provider imports outside the gateway.

- **PostToolUse (Edit|Write):** `.claude/scripts/post-edit-check.sh` — runs ruff/mypy/pytest on api, lint/tsc on web (permissive — never blocks the edit).

If a rule isn't here, check `.claude/rules/`. If you're scaffolding new structure, check `.claude/skills/`. If you need the full why, check `docs/PRD.md`. If you need to escalate, see `.claude/rules/incident-response.md`.